

# LAPORAN PRAKTIKUM BIG DATA

## Praktikum 4



OLEH :

JARDEL POLIVIERA

2411533012

DOSEN PENGAMPU:

AFDHAL DINILHAK. M.Kom

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2026

Link repo :

[https://github.com/JardelPoliviera/BigData26\\_B\\_2411533012\\_JardelPoliviera](https://github.com/JardelPoliviera/BigData26_B_2411533012_JardelPoliviera)

Link gdrive :

[https://drive.google.com/drive/folders/1Gjtd2beuucuf3Hwnllp6nuvtqGo\\_cz\\_O?usp=sharing](https://drive.google.com/drive/folders/1Gjtd2beuucuf3Hwnllp6nuvtqGo_cz_O?usp=sharing)

## A. PENDAHULUAN

Analisis mobilitas perkotaan modern menuntut pemahaman yang mendalam mengenai pola pergerakan masyarakat setiap harinya. Salah satu indikator terbaik untuk mengukur aktivitas ini adalah data operasional taksi kuning yang merekam jutaan perjalanan penumpang secara rinci. Data rekaman perjalanan ini tidak sekadar berisi angka, melainkan menyimpan informasi berharga terkait jam sibuk, jarak tempuh, hingga fluktuasi permintaan layanan transportasi komersial di suatu kota.

Studi kasus ini berfokus pada perbandingan komprehensif antara dataset operasional taksi pada bulan Januari dan Februari pada tahun yang sama. Tujuan utama dari analisis komparatif ini adalah untuk mengidentifikasi apakah terdapat pergeseran tren perilaku komuter atau justru menunjukkan pola yang konsisten dari waktu ke waktu. Melalui serangkaian proses ekstraksi, pembersihan data, hingga pemetaan visual menggunakan bahasa pemrograman Python, tahapan analisis ini dirancang untuk menyingkap wawasan penting di balik tingginya volume transaksi harian..

## B. TUJUAN

1. Mahasiswa dapat menyiapkan notebook Colab yang siap pakai dan mendokumentasikan spesifikasi lingkungan kerjanya (Python version, library version, RAM, disk).
2. Mahasiswa dapat mengunduh dataset publik secara terprogram dan memeriksa integritas awalnya (ukuran file, jumlah baris, skema kolom).
3. Mahasiswa dapat mengukur pemakaian memori sebuah DataFrame dan menurunkannya melalui pemilihan kolom serta optimasi tipe data.
4. Mahasiswa dapat memproses file yang lebih besar dari RAM yang tersedia menggunakan teknik chunking
5. Mahasiswa dapat menghasilkan tabel agregasi (per jam, per metode pembayaran) dan menyimpannya dalam format CSV serta Parquet.
6. Mahasiswa dapat menjalankan kueri agregasi menggunakan PySpark local mode dan menjelaskan perbedaan konseptualnya dengan pandas.

## C. LANGKAH PRAKTIKUM

### 1. K1

```
*** Requirement already satisfied: q in /usr/local/lib/python3.13/dist-packages (2.7)
Requirement already satisfied: duckdb in /usr/local/lib/python3.13/dist-packages (1.3.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.13/dist-packages (3.7)
Requirement already satisfied: shapely in /usr/local/lib/python3.13/dist-packages (2.1.2)
Requirement already satisfied: tinydb in /usr/local/lib/python3.13/dist-packages (4.9.0)
Requirement already satisfied: deltalake>1.0 in /usr/local/lib/python3.13/dist-packages (1.6.6)
Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3.13/dist-packages (from shapely) (2.1.3)
Requirement already satisfied: arro3-core>=0.5.0 in /usr/local/lib/python3.13/dist-packages (from deltalake>1.0) (0.5.0)
Requirement already satisfied: deprecated>=1.2.18 in /usr/local/lib/python3.13/dist-packages (from deltalake>1.0) (3.0.0)
Requirement already satisfied: wrapt<3,>=1.16 in /usr/local/lib/python3.13/dist-packages (from deprecated>=1.2.18->del) (1.16.0)
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
[ baca trips_bersih ] 1.8412 s
baris: 2,986,910 | kolom: 18
```

Kode pada bagian ini digunakan untuk mempersiapkan lingkungan kerja dengan menginstal pustaka yang dibutuhkan seperti DuckDB, NetworkX, Shapely, TinyDB, dan DeltaLake. Selanjutnya, kode menyambungkan Google Colab dengan Google Drive dan memastikan direktori data bersih (trips\_bersih/) dari praktikum sebelumnya tersedia. Setelah direktori siap, fungsi ukur didefinisikan secara khusus untuk mencatat waktu eksekusi dari setiap proses yang akan dilakukan. Data kemudian dibaca dari format Parquet menggunakan pustaka pandas, kemudian nama-nama kolom aslinya diubah menjadi nama yang lebih mudah dipahami secara lokal, dan sistem mengambil sampel acak hingga 300.000 baris yang akan dipakai pada simulasi-simulasi selanjutnya agar waktu komputasinya tidak terlalu berat.

### 2. K-2. Tabel Biasa vs Key-value

```
[relasional: muat tabel] 2.0866 s
[key-value: bangun dict] 5.3034 s
jumlah key: 300000
[cari 200 perjalanan SQL tanpa indeks] 7.1963 s
[cari 200 perjalanan SQL dengan indeks] 0.0044 s
[cari 200 perjalanan key-value] 0.0015 s
hasil sama? True
[hitung per wilayah SQL GROUP BY] 0.2027 s
[hitung per wilayah key-value (buka semua nilai)] 1.3240 s
[('Manhattan', 267034, 22.7), ('Queens', 27288, 70.34), ('Unknown', 3787, 39.13)]
hasil sama? True
```

Kode ini membandingkan waktu yang dibutuhkan antara penyimpanan berbasis tabel relasional dan penyimpanan model basis data key-value. Data diolah dengan dua cara secara paralel: dimasukkan ke dalam basis data ringan (SQLite) dan dirangkai ulang menjadi bentuk dictionary JSON di mana nilai diakses menggunakan id\_perjalanan. Eksperimen pertama memperlihatkan bahwa pencarian identitas tertentu menggunakan metode relasional akan bekerja ribuan kali jauh lebih cepat ketika pengembang mengaplikasikan struktur indeks tabel (CREATE INDEX) yang kinerjanya menyamai metode key-value. Namun pada eksperimen kedua tentang perhitungan agregasi atas semua data, model key-value justru tertinggal sangat lambat lantaran program terpaksa membuka sistem JSON untuk tiap satu sel datanya, berlawanan dengan kepraktisan SQL.

### 3. K-3. Document: Bentuk Data yang Fleksibel

```
... jumlah dokumen: 20000
{
  "id_perjalanan": 1,
  "waktu_jemput": "2023-01-16 10:40:39",
  "jemput": {
    "zona": 48,
    "wilayah": "Manhattan"
  },
  "tujuan": {
    "zona": 164
  },
  "biaya": {
    "jarak_mil": 1.55,
    "total": 19.32,
    "metode_bayar": "Kartu kredit",
    "tip": 3.22
  }
}
jemput di Manhattan & total > 50: 693
dokumen yang punya field tip : 15906
dokumen yang TIDAK punya tip : 4094
dokumen yang punya field cuaca : 1
total dokumen sekarang : 20001
```

Model pengelolaan basis data dokumen diperagakan melalui pustaka TinyDB, tempat skema direpresentasikan dalam blok-blok dokumen (document store) JSON yang dapat bersarang. Keunggulan skema ini terlihat dari strukturnya yang dapat dimanipulasi dengan leluasa; contohnya kolom tip tidak disamaratakan pada seluruh data, melainkan hanya diaktifkan bila pelanggan menggunakan metode bayar menggunakan sistem kartu kredit. Kode tersebut turut membuktikan manfaat kelenturan data melalui penyelipan dan penambahan berkas informasi baru secara otomatis (berupa bidang kategori cuaca) pada dokumen terakhir yang ditambahkan ke koleksi, tanpa mewajibkan pengembang mengubah format struktur dasar dari dokumen perjalanan yang sudah tersimpan layaknya migrasi kolom ALTER TABLE pada sistem tabel relasional.

### 4. K-4. Penyimpanan per Kolom: DuckDB pada Parquet

```
... [DuckDB hitung per wilayah (langsung dari Parquet)] 0.3343 s
      wilayah_jemput  jumlah  rata_bayar
0      Manhattan    2659609    22.72
1      Queens       270315    70.46
2      Unknown      37609     39.24
3      Brooklyn     15724     32.72
4      Bronx        3074      34.44
5  __HIVE_DEFAULT_PARTITION__  340     74.72
6      Staten Island    211     74.07
7      EWR           28      106.41
[SQLite muat seluruh baris ke tabel] 12.2644 s
[SQLite hitung per wilayah] 1.9939 s
hasil sama? False
[DuckDB hanya wilayah Staten Island] 0.0624 s
[(211, 74.07)]
Parquet (18 kolom): 66.5 MB | SQLite (5 kolom): 114.2 MB
```

Untuk analisis perhitungan atas berjuta baris kolom secara paralel, penyimpanan berbasis per kolom dieksekusi memanfaatkan interaksi DuckDB yang langsung membaca arsip file Parquet tanpa terlebih dahulu mereplika datanya (CREATE VIEW). Hasil eksperimen menunjukkan bahwa DuckDB melaksanakan analisis rata-rata pengelompokan yang identik namun secara eksponensial (belasan kali) jauh melampaui waktu eksekusi basis data SQLite yang memeriksa per baris. Peningkatan kecepatan mutlak ini disebabkan oleh operasi khusus, seperti memangkas kolom yang tidak terlibat perihal hitungan (column pruning) serta mereduksi dan menolak mesin memuat folder dari wilayah zona yang sama sekali tidak dicari ketika filter kondisi spesifik sedang dijalankan (partition pruning).

## 5. K-5. Graph: Pertanyaan tentang Keterhubungan

```
[hitung arus antar zona] 0.4693 s
node (zona): 262 edge (arus): 21583
graph: [(237, 236, 22065), (236, 237, 18810), (236, 236, 14110), (237, 237, 13753), (264, 264, 13562)]
SQL : [(237, 236, 22065), (236, 237, 18810), (236, 236, 14110), (237, 237, 13753), (264, 264, 13562)]
zona tujuan terpopuler: [(236, 144209), (237, 130369), (161, 113991), (230, 87706), (170, 87110)]
[2 langkah graph (NetworkX)] 0.0067 s
[2 langkah SQL self-join (DuckDB)] 0.0181 s
zona awal: 236 terjangkau (graph): 214 | (SQL): 214 sama? True
dalam <= 1 langkah: 74 zona
dalam <= 2 langkah: 214 zona
dalam <= 3 langkah: 214 zona
dalam <= 4 langkah: 214 zona
```

Implementasi pemetaan tata letak transportasi dalam wujud format relasi basis data graph (jaringan) yang diproses memakai pustaka NetworkX terlihat pada bagian skrip ini. Tiap-tiap area zona jemput dan zona tujuan digambarkan melalui sistem node dan dihubungkan secara topologi oleh pergerakan interaktif atau edge yang menyematkan informasi volume lintas perjalanan sebagai kalkulasi bebannya. Dibandingkan dengan manipulasi sistem agregasi standar tabel (relasional atau SQL) yang mendikte pengembang untuk senantiasa menciptakan siklus gabungan berulang (self-join) setiap kali perlu berekspansi ke lompatan area analisis, cara interogasi model graph menyediakan alternatif jawaban keterhubungan yang lebih ringkas dan otomatis tanpa mengorbankan durasi efisiensi pencarian.

## 6. K-6. Indeks Spasial

```
** [cara 1 loop Python, 5 kotak] 12.9721 s
[cara 2 NumPy, 1000 kotak] 0.5644 s
[bangun indeks STRtree] 0.0787 s
[cara 3 STRtree, 1000 kotak] 0.0081 s
5 kotak pertama loop: [0, 0, 0, 0, 0] | STRtree: [0, 0, 0, 0, 0]
hasil NumPy dan STRtree sama? True | loop dan STRtree sama (5 pertama)? True
total titik ditemukan (1000 kotak): 0
```

Melalui pengolahan indeks spasial menggunakan R-tree (jenis STRtree milik pustaka Shapely), sistem sanggup dengan akurat mencari rentang radius atau keberadaan spesifik suatu obyek dari besarnya muatan angka spasial pada radius bumi. Dengan mencetak pola ribuan target geometri dalam bentuk titik-titik koordinat virtual (longitude dan latitude), simulasi perbandingan waktu pencarian dipertemukan melalui penggunaan loop manual, NumPy secara bersamaan, serta

indeks spasial berjenjang dari STRtree. Penentuan letak indeks spasial ini merangkum kumpulan titik koordinat yang dekat lokasinya di dalam pola kotak-kotak peta bersinggungan sehingga komputer berpeluang menghemat masa pelacakan secara mendadak (ratusan kali lebih kilat) melewati setiap area tak relevan dari ruang sasaran seolah mengabaikan area buangan.

## 7. K-7. Delta Lake: Versi, Time Travel, dan Schema Enforcement

```
versi: 0 | baris: 100000  
versi: 1 | baris: 150000  
baris yang akan dikoreksi: 20  
versi: 2  
baris v0: 100000 | v1: 150000 terbaru: 150000  
selisih total_bayar (terbaru - v1): 20.0 jumlah baris dikoreksi: 20  
2 UPDATE  
1 WRITE  
0 WRITE  
['00000000000000000000000000000000.json', '00000000000000000000000000000001.json', '00000000000000000000000000000002.json']  
append kolom asing DITOLAK: SchemaMismatchError  
versi setelah percobaan gagal: 2
```

Kode di atas memperkenalkan pemrosesan penyimpanan dengan sistem log transaksi berstandar reliabel dari ACID menggunakan fondasi model data Delta Lake. Implementasi operasi mutasi yang dijalankan pada program ini mencakup prosedur penulisan keseluruhan berkas data awal (versi 0), penambahan 50.000 rekaman (versi 1), serta pembaruan (update) kompensasi biaya ongkos wilayah secara eksklusif (versi 2) tanpa memotong versi terdahulunya. Oleh karena modifikasinya berseri dan menghasilkan penambahan arsip transaksi berekstensi JSON di dalam ruang lokal log secara terus-menerus, sistem dapat kapan saja mereset rekaman bacaan secara spesifik menyesuaikan keadaan pada lampau waktu (time travel) tanpa ada selisih kekeliruan. Disiplin dan keketatan Delta Lake turut mencakup penolakan langsung terhadap baris dokumen (berupa `SchemaMismatchError`) jika terdeteksi mengusung jenis properti format tak wajar, demi mencegah kegagalan dan menjaga keluhuran versi data sistem.

### 8. K-8. Small File Problem pada Tabel Delta

```
... file data aktif sebelum: 1 | versi: 2
file data aktif setelah 20 append kecil: 21 | versi: 22 | baris: 160000
hasil compact: {'numFilesAdded': 1, 'numFilesRemoved': 21}
file data aktif setelah compact: 1 | versi: 23 | baris: 160000
waktu baca sebelum compact: 0.0833 s | sesudah: 0.0557 s
file parquet di disk: 24
file yang akan dihapus VACUUM (dry run): 23
```

Pada skrip di tahap ini, dicontohkan pemecahan penanggulangan permasalahan jumlah file yang membengkak karena pencetakan rekam data tambahan yang sangat kecil dan berlanjut secara sporadis (small file problem). Berbagai versi log penambahan data skala kecil ini dievaluasi serta digabungkan oleh eksekusi fungsi perampingan compact(), sehingga file data yang aktif menjadi jauh lebih ramping, mudah dan stabil sewaktu diloading ulang namun tersimpan utuh di versi paling baru. Namun, bekas versi lama pada direktori aslinya tidak dihapus agar histori

time travel sebelumnya dapat dilestarikan, untuk memutlakkan pencabutan fragmen masa lalu dan melegakan disk, developer diwajibkan menjalankan program aksi VACUUM.

#### 9. K-9. Merangkum dan Memilih

| ... | langkah                                           | detik   |
|-----|---------------------------------------------------|---------|
|     | baca trips_bersih                                 | 1.8412  |
|     | relasional: muat tabel                            | 2.0866  |
|     | key-value: bangun dict                            | 5.3034  |
|     | cari 200 perjalanan SQL tanpa indeks              | 7.1963  |
|     | cari 200 perjalanan SQL dengan indeks             | 0.0044  |
|     | cari 200 perjalanan key-value                     | 0.0015  |
|     | hitung per wilayah SQL GROUP BY                   | 0.2027  |
|     | hitung per wilayah key-value (buka semua nilai)   | 1.3240  |
|     | DuckDB hitung per wilayah (langsung dari Parquet) | 0.3343  |
|     | SQLite muat seluruh baris ke tabel                | 12.2644 |
|     | SQLite hitung per wilayah                         | 1.9939  |
|     | DuckDB hanya wilayah Staten Island                | 0.0624  |
|     | hitung arus antar zona                            | 0.4693  |
|     | 2 langkah graph (NetworkX)                        | 0.0067  |
|     | 2 langkah SQL self-join(DuckDB)                   | 0.0181  |
|     | cara 1 loop Python, 5 kotak                       | 12.9721 |
|     | cara 2 NumPy, 1000 kotak                          | 0.5644  |
|     | bangun indeks STRtree                             | 0.0787  |
|     | cara 3 STRtree, 1000 kotak                        | 0.0081  |

Di bagian penghujung modul, kode mengumpulkan semua angka komparasi durasi waktu yang sudah disimulasikan dari setiap variasi jenis penyimpanan. Hasilnya diekspor memakai integrasi pustaka data (pandas) agar disimpan di media disk pengguna dengan wujud tabel ringkasan (dataframe dan format CSV pada benchmark\_nosql.csv) yang nanti bisa dijadikan evaluasi patokan untuk mendesain platform Big Data yang mumpuni di laporan praktikum mandiri.

#### D. Analisis Hasil

1. Klaim & Angka: Tabel tanpa indeks (7,1963 detik) sekitar 1.635 kali lebih lambat dibandingkan tabel berindeks (0,0044 detik). Tabel berindeks (0,0044 detik) hanya sekitar 2,9 kali lebih lambat dibandingkan sistem key-value (0,0015 detik).

Alasan: Perbedaan yang benar-benar berarti (signifikan) adalah selisih antara kueri tanpa indeks dan berindeks. Tanpa indeks, sistem harus memeriksa ketersediaan id baris demi baris, sementara indeks memberikan semacam daftar isi instan. Selisih antara berindeks dan key-value (sekian milidetik) tidak terlalu terasa perbedaannya untuk pencarian dalam skala ini di memori lokal.

2. Klaim & Angka: Penyimpanan key-value (1,3240 detik) sekitar 6,5 kali lebih lambat dibandingkan SQL GROUP BY (0,2027 detik) untuk menghitung rata-rata.

Alasan: Pada sistem key-value, nilai data hanya disimpan sebatas teks. Untuk melakukan operasi perhitungan agregasi/rata-rata, program terpaksa harus memuat (json.loads) dan membongkar dokumen teks setiap nilainya terlebih dahulu untuk mencari field target. Sementara itu, SQL maupun penyimpanan kolom sudah mengenali kolom yang berupa angka, sehingga bisa melakukan kalkulasi matematik secara seketika.

3. Klaim (Keuntungan): Kita dapat langsung memasukkan atau menambahkan field spesifik/baru (seperti kolom cuaca pada dokumen ke-20.001) secara mandiri tanpa mewajibkan kita mengubah struktur skema secara masal (ALTER TABLE) maupun menyisipkan nilai kosong/NULL untuk keseluruhan 20.000 dokumen lainnya. Klaim (Risiko): Fleksibilitas ini mendatangkan risiko ketiadaan validasi pakem/penegakan skema dari pihak database. Bila aplikasi mengalami kesalahan pengetikan nama field data, basis data tetap akan menerima input secara serampangan, sehingga kebersihan data tidak terjamin.
4. Klaim & Angka: Pada data Anda, DuckDB (0,3343 detik) sekitar 6 kali lebih cepat dibandingkan dengan SQLite (1,9939 detik) untuk eksekusi perhitungan. Alasan (1 - Terkait Format Kolom): Terjadinya proses column pruning, yaitu DuckDB mampu melewati bagian kolom yang tak dipakai dengan mengabaikannya dan hanya berfokus menyoroti target hitungan. Alasan (2 - Bukan Format Kolom): Kinerja DuckDB secara khusus memang dirancang berjenis vectorized yang menunjang pemrosesan bacaan kueri dari banyak kolom/baris dalam satu waktu (multithreading) bersamaan dengan pendayagunaan banyak core dari mesin, tidak seperti barisan eksekusi SQLite konvensional.
5. Klaim: Sekalipun hasil NetworkX dan DuckDB sama-sama di kisaran sepersekian milidetik pada penelusuran 2 langkah, NetworkX (graph) tetap layak dipilih karena keluwesan eksplorasi pertanyaannya yang dapat dikendalikan sesuka hati tanpa memperumit perintah sintaksis (terbukti dari pergerakan bebas  $< 4$  k-langkah = 214 zona terjangkau yang dikalkulasi praktis dalam graph).

Alasan: Pada basis data relasional/SQL, pencarian keterhubungan langkah ke target selanjutnya mewajibkan aplikasi menulis sintaks penggabungan tabel (self-join) baru. Jika pencarian ingin diperluas ke 4 langkah ( $k=4$ ), Anda perlu menulis fungsi iterasi self-join beruntun sebanyak tiga atau empat kali yang justru rentan dan rumit ditulis manusia. Dalam graph, mencari hingga beberapa k-langkah cukup mengubah variabel cutoff parameter

6. Klaim & Angka: Pembanding Loop (12,9721 detik) hanya membaca 5 kotak; apabila dirasionalkan (linier) ke 1000 kotak, eksekusinya memerlukan 2594 detik. Dibandingkan dengan STRtree (0,0081 detik), STRtree lebih dari 320 ribu kali lebih cepat ketimbang Loop Python. Namun, apabila dikomparasikan terhadap

pemrosesan 1000 vektor dari NumPy (0,5644 detik), indeks STRtree "hanya" 70 kali lebih cepat.

Alasan Perbedaan & Arti Penulisan Klaim: Angka ini berbeda drastis karena metode pertama (pemeriksaan satu per satu/ Loop Python) murni tidak berfokus pada kecepatan, sedangkan metode kedua (NumPy) menangani deretan perhitungan secara vectorized/paralel sehingga ia tampil jauh lebih efisien dibanding looping telanjang. Artinya, perumusan sebuah klaim "Indeks X ratusan ribu kali lebih cepat" adalah bias dan menyesatkan. Agar klaim performa sah, ia harus mengikutsertakan garis batas dasar (baseline) pembandingan target aslinya.

7. Klaim (a) Pembaruan tidak menghilangkan data: Dari output, fungsi time travel (v1) memungkinkan program kita memulihkan nilai ke waktu dan volume (len 150000) yang persis sama. Delta Table membaca ketersediaan tabel v1 dengan selisih nominal bayar (20.0) yang sesuai jumlah baris dikoreksi, membuktikan pembaruan mutasi 2 tidak merusak nilai dokumen di waktu 1. Klaim (b) Gagal tak berjejak: Upaya penyuntikan 1 kolom asing ditolak telak dan disetop (SchemaMismatchError) tanpa mencatat versi setengah jalan, mempertahankan tabel seutuhnya di "versi: 2" yang steril dari sisa gangguan error. Alasan File Compaction (24 vs 1): Walau versi compaction sudah disatukan ke dalam 1 file aktif, berkas 24 lama di memori internal/disk tidak serta-merta dihapuskan otomatis, alasannya demi memelihara catatan log riwayat apabila di lain waktu aplikasi membutuhkan prosedur pengembalian data (time travel). Berkas-berkas usang ini akan ditimbun di `_delta_log/` hingga instruksi VACUUM sengaja dijalankan.

## E. Studi Kasus

| Fitur Aplikasi                                                                                       | Cara Menyimpan yang Dipilih                         | Alasan (Berdasarkan Angka Benchmark)                                                                                                                                                                                                                                          | Kelemahan yang Harus Diwaspadai                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Detail Perjalanan (Respons Cepat) Halaman detail yang dibuka penumpang berdasarkan id perjalanan. | Key-Value (seperti kamus/dict)                      | Berdasarkan hasil uji K-2, pencarian id acak dengan model key-value menjadi yang tercepat dengan waktu 0,0015 detik. Hasil ini mutlak menjamin respons instan berskala milidetik, mengalahkan SQL tanpa indeks (7,1963 detik) maupun dengan indeks (0,0044 detik).            | Sangat lambat jika sewaktu-waktu aplikasi memerlukan kalkulasi angka atas seluruh rekaman, karena sistem memakan waktu lama (1,3240 detik) untuk membongkar (loads) teks JSON satu per satu.                |
| 2. Dashboard Pendapatan Menghitung pendapatan per wilayah per jam dengan filter.                     | Penyimpanan per Kolom (DuckDB + Parquet berpartisi) | Terbukti unggul mutlak dalam kalkulasi agregasi (column pruning) dengan menyelesaikan fungsi rata-rata berdasar wilayah dalam 0,3343 detik. Keberadaan partisi (seperti filter wilayah Staten Island) mempercepat eksekusinya menjadi hanya 0,0624 detik (partition pruning). | Format per kolom tidak ideal untuk pembaharuan (update) atau penyisipan baris tunggal secara real-time dengan frekuensi tinggi, karena akan mereplikasi ulang keseluruhan blok file.                        |
| 3. Rekomendasi Rute & Zona Zona sering dituju & zona yang dapat dicapai dalam 2-3 langkah.           | Jaringan (Graph) (NetworkX)                         | Sanggup menyelesaikan pencarian rute lintas 2-langkah hanya dalam 0,0067 detik. Walau performanya sepadan dengan self-join DuckDB (0,0181 detik), representasi graph tidak memerlukan iterasi rumit per penambahan lompatan wilayah (hanya mengubah limit k batas algoritma). | Mengharuskan struktur relasi antarzona (keseluruhan nodes dan edges) berada dan termuat penuh di dalam RAM memori agar bisa dihitung dengan kilat.                                                          |
| 4. Pencarian Radius Lokasi Taksi Mencari riwayat taksi menjemput dalam radius 500 meter.             | Indeks Spasial (STRtree)                            | Indeks R-Tree secara cerdas memangkas proses dan menyelesaikan pemetaan 1.000 titik radius kotak dalam 0,0081 detik. Hasil ini puluhan hingga ribuan kali lipat lebih efisien daripada algoritma loop manual (12,9721 detik) atau agregasi NumPy (0,5644 detik).              | Indeks spasial hanya memberikan keuntungan absolut pada parameter saring/filter lokasi wilayah murni. Jika algoritma turut difilter atribut non-geografis lain secara banyak, sistem akan kembali melambat. |
| 5. Audit Keuangan (Rollback) Kembalikan tabel tarif ke keadaan                                       | Tabel Delta Lake                                    | Karena tabel Delta menggunakan log transaksi ACID, semua riwayat tetap                                                                                                                                                                                                        | Karena tidak ada versi data lama yang dihapus ketika data diperbarui,                                                                                                                                       |

|                                      |  |                                                                                                                                                                                                                                        |                                                                                                                                                     |
|--------------------------------------|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| tanggal 1 (saat ada koreksi keliru). |  | terjaga utuh. Kemampuan Time Travel atau pemanggilan versi ke-n memungkinkan kita membaca versi tabel sesaat sebelum koreksi atau merekonstruksi <code>dt.restore(verisi_lawas)</code> dengan seketika tanpa mencemari tabel berjalan. | berkas sistem log (Small File Problem) akan menumpuk dan membengkakkan disk penyimpan jika tidak rutin dieksekusi pembersihan <code>VACUUM</code> . |
|--------------------------------------|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

## F. Latihan

```

# Menyiapkan 2.000 ID acak
random.seed(42)
id_uji_2000 = random.sample(range(1, len(data) + 1), 2000)

def cari_sql_2000():
    cur = con.cursor()
    return [cur.execute("SELECT * FROM perjalanan WHERE id_perjalanan = ?", (i,)).fetchone() for i in id_uji_2000]

def cari_kv_2000():
    return [json.loads(kv[f"perjalanan:{i}"]) for i in id_uji_2000]

# Pastikan indeks sudah dibuat dari sel K-2 sebelumnya
waktu_sql = ukur("SQL (indeks) 2000 pencarian", cari_sql_2000)
waktu_kv = ukur("Key-value 2000 pencarian", cari_kv_2000)

# Menghitung rata-rata dalam mikrodetik
waktu_total_sql = catatan[-2]["detik"]
waktu_total_kv = catatan[-1]["detik"]

print(f"Rata-rata SQL (indeks): {(waktu_total_sql / 2000) * 1_000_000:.2f} mikrodetik/pencarian")
print(f"Rata-rata Key-value : {(waktu_total_kv / 2000) * 1_000_000:.2f} mikrodetik/pencarian")

... [SQL (indeks) 2000 pencarian] 0.0839 s
[Key-value 2000 pencarian] 0.0316 s
Rata-rata SQL (indeks): 41.95 mikrodetik/pencarian
Rata-rata Key-value : 15.80 mikrodetik/pencarian
[SQL (indeks) 2000 pencarian] 0.0606 s
[Key-value 2000 pencarian] 0.0190 s
Rata-rata SQL (indeks): 30.30 mikrodetik/pencarian
Rata-rata Key-value : 9.50 mikrodetik/pencarian

```

Pencarian menggunakan key-value secara teori akan tetap sedikit lebih unggul atau setara dengan tabel berindeks karena kuncinya langsung mengarah ke letak data di memori (seperti direktori hash). Pada perhitungan mikrodetik, perbedaannya menunjukkan efisiensi pencarian titik tunggal tanpa perlu melakukan pemindaian tabel penuh.

```

Latihan 2: Perbandingan Pencarian Dokumen (TinyDB dan Tabel (SQLite))

# Misal kita mencari zona tujuan 164 dengan metode bayar Tunai
zona_target = 164

# Cara TinyDB (K-3)
Q = Query()
hasil_tinydb = tabel.search((Q.tujuan.zona == zona_target) & (Q.biaya.metode_bayar == "Tunai"))
print(f"Jumlah TinyDB: {len(hasil_tinydb)}")

# Cara SQL SQLite (K-2)
hasil_sql = con.execute("SELECT COUNT(*) FROM perjalanan WHERE zona_tujuan = ? AND metode_bayar = 'Tunai'", (zona_target,))
print(f"Jumlah SQL : {hasil_sql}")

... Jumlah TinyDB: 93
Jumlah SQL : 1086

```

Hasil perhitungan keduanya tidak akan sama. Sebab utamanya terletak pada ukuran sampel yang dimuat ke dalam basis data. Pada langkah K-2, tabel SQLite perjalanan dimuat menggunakan seluruh variabel sampel (`sample[KOLOM].copy()`) yang berjumlah hingga 300.000 baris. Sebaliknya, pada langkah K-3, TinyDB hanya disuntikkan sebagian kecil dokumen, yaitu `sample.head(20_000)`.

Latihan 3: Column Pruning pada DuckDB

```
sql_1_kolom = "SELECT SUM(total_bayar) FROM perjalanan"
sql_3_kolom = "SELECT SUM(total_bayar), SUM(jarak_mil), AVG(zona_jemput) FROM perjalanan"

ukur("DuckDB SUM 1 kolom", lambda: con_d.sql(sql_1_kolom).df())
ukur("DuckDB SUM 3 kolom", lambda: con_d.sql(sql_3_kolom).df())
```

... [DuckDB SUM 1 kolom] 0.2483 s  
[DuckDB SUM 3 kolom] 0.3970 s

|   | sum(total_bayar) | sum(jarak_mil) | avg(zona_jemput) |
|---|------------------|----------------|------------------|
| 0 | 8.162144e+07     | 10231889.61    | 166.544273       |

Waktu eksekusi DuckDB untuk kueri 1 kolom akan lebih cepat dibandingkan kueri 3 kolom sekaligus. DuckDB beroperasi langsung membaca arsip file berformat Parquet, yang merupakan tipe penyimpanan per kolom. Saat kueri hanya meminta satu kolom (seperti `total_bayar`), DuckDB mengaplikasikan column pruning dengan melewati dan tidak membaca bongkahan data dari kolom lain. Menambah kolom untuk dihitung (SUM) memaksa mesin membaca lebih banyak blok memori dari disk.

Latihan 4: Menghitung Zona Tujuan Berbeda (Out-degree Graph vs SQL)

```
# 1. Cara Graph: Hitung tetangga terhubung dari setiap node (zona tujuan berbeda = jumlah successors)
out_degrees = [(n, len(list(G.successors(n)))) for n in G.nodes()]
top_5_graph = sorted(out_degrees, key=lambda x: (-x[1], x[0]))[:5]
print("Top 5 Graph (Out-degree):", top_5_graph)

# 2. Cara SQL: Hitung COUNT(DISTINCT) tujuan dari tabel arus
top_5_sql = con_d.sql("""
    SELECT asal, COUNT(DISTINCT tujuan) AS jml_tujuan
    FROM arus
    GROUP BY asal
    ORDER BY jml_tujuan DESC, asal ASC
    LIMIT 5
""").fetchall()
print("Top 5 SQL : ", top_5_sql)
```

... Top 5 Graph (Out-degree): [(132, 260), (138, 254), (264, 241), (230, 234), (140, 233)]  
Top 5 SQL : [(132, 260), (138, 254), (264, 241), (230, 234), (140, 233)]

Pada struktur relasi graph, menghitung jumlah koneksi cabang ke wilayah berbeda cukup menggunakan fitur bawaan jumlah simpul yang mengarah keluar (out-degree) karena setiap edge unik merepresentasikan satu pasang hubungan asal-tujuan yang ada di data.

```

# Menguji dengan sisi kotak 0,02 derajat dan 0,001 derajat
for ukuran in [0.02, 0.001]:
    kotak_baru = [box(x - ukuran, y - ukuran, x + ukuran, y + ukuran) for x, y in zip(cx, cy)]

    def numpy_baru():
        hasil = []
        for x, y in zip(cx, cy):
            m = (bujur >= x - ukuran) & (bujur <= x + ukuran) & (lintang >= y - ukuran) & (lintang <= y + ukuran)
            hasil.append(int(m.sum()))
        return hasil

    ukur(f"NumPy kotak {ukuran}", numpy_baru)
    ukur(f"STRtree kotak {ukuran}", lambda: [len(pohon.query(k, predicate="contains")) for k in kotak_baru])

... [NumPy kotak 0.02] 0.5889 s
[STRtree kotak 0.02] 0.0079 s
[NumPy kotak 0.001] 0.6170 s
[STRtree kotak 0.001] 0.0282 s

```

Indeks spasial bekerja dengan mengelompokkan titik-titik koordinat spasial ke dalam beberapa kelompok batas pembungkus berjenjang. Ketika ukuran wilayah yang dicari (kotak) semakin diperkecil menjadi 0,001 derajat, persinggungan antara kotak pencarian dengan kotak kelompok di memori menjadi lebih sedikit. Akibatnya, R-Tree mengabaikan lebih banyak blok pencarian yang tidak relevan. Oleh karena itu, rasio percepatan STRtree dibandingkan algoritma sekuensial (NumPy) akan semakin besar dan memuncak pada ukuran kotak terkecil.

```

Latihan 6: Time Travel vs Restore di Delta Lake

# Pastikan PATH_DELTA sudah mengarah ke lokal atau '/content/trips_delta'
# seperti pada K-7 dan K-8 Anda
dt = DeltaTable(PATH_DELTA_LOKAL)
print("versi sebelum restore:", dt.version(), "| baris:", len(dt.to_pandas()))

# Kembalikan tabel ke keadaan versi 1
dt.restore(1)

dt = DeltaTable(PATH_DELTA_LOKAL)
print("versi setelah restore:", dt.version(), "| baris:", len(dt.to_pandas()))
print([h.get("operation") for h in dt.history()][3])

... versi sebelum restore: 23 | baris: 160000
versi setelah restore: 24 | baris: 150000
['RESTORE', 'OPTIMIZE', 'WRITE']

```

Time travel hanyalah aksi membaca (read-only) keadaan arsip di masa lalu tanpa memengaruhi wujud basis data berjalan. Sebaliknya, restore adalah operasi menulis tabel ulang yang aktif. Tabel akan ditarik mundur wujudnya ke versi sebelumnya (misalnya V1), tetapi perubahan mundur ini tidak akan menghapus riwayat V2 dan V3. Pemulihan ini akan dibukukan sebagai transaksi RESTORE baru di `_delta_log/` (menjadi versi 24 pada sistem yang melalui K-8), sehingga riwayat log linear tetap mutlak terjaga dan ACID tidak dilanggar.

## G. Tugas Praktikum

1. Rekap benchmark. Kumpulkan `benchmark_nosql.csv` dan tabel rekap K-9 yang sudah terisi, dengan satu kalimat interpretasi untuk tiap baris.
  - 1) baca `trips_bersih` (1.8412 s): Waktu yang dibutuhkan pustaka pandas untuk memuat dataset Parquet ke dalam memori komputer cukup cepat berkat format datanya yang efisien.

- 2) relasional: muat tabel (2.0866 s): Menyalin dataframe ke dalam tabel relasional SQLite memakan waktu sekitar dua detik karena data harus ditulis ulang ke format baris.
- 3) key-value: bangun dict (5.3034 s): Pembuatan kamus key-value memakan waktu paling lama di tahap persiapan karena program harus mengurai setiap baris menjadi teks (JSON) satu per satu.
- 4) cari 200 perjalanan SQL tanpa indeks (7.1963 s): Pencarian ini menjadi operasi paling lambat karena basis data terpaksa memindai tabel secara buta dari baris pertama hingga terakhir.
- 5) cari 200 perjalanan SQL dengan indeks (0.0044 s): Keberadaan indeks membuat pencarian melonjak ribuan kali lebih cepat karena sistem langsung melompat ke alamat memori yang tepat.
- 6) cari 200 perjalanan key-value (0.0015 s): Key-value menjadi juara dalam pencarian data tunggal karena kunci id bertindak langsung sebagai pemeta absolut ke lokasi data.
- 7) hitung per wilayah SQL GROUP BY (0.2027 s): Operasi agregasi SQLite cukup cepat karena basis data mengenali kolom angka secara native.
- 8) hitung per wilayah key-value (buka semua nilai) (1.3240 s): Waktu agregasi melambat drastis karena program membuang waktu membuka struktur teks JSON pada tiap baris sebelum bisa mengambil nilai angkanya.
- 9) DuckDB hitung per wilayah (langsung dari Parquet) (0.3343 s): DuckDB menghitung agregasi sangat gesit langsung dari file Parquet murni karena memotong kolom tak relevan (column pruning).
- 10) SQLite muat seluruh baris ke tabel (12.2644 s): Menyimpan jutaan data penuh ke tabel relasional disk menguras waktu belasan detik, menjadikannya tahap terberat secara I/O.
- 11) SQLite hitung per wilayah (1.9939 s): Perhitungan agregasi SQLite atas seluruh baris memakan waktu hampir 2 detik, lebih lambat dibandingkan engine DuckDB.
- 12) DuckDB hanya wilayah Staten Island (0.0624 s): Pemfilteran spesifik wilayah berjalan seketika karena partition pruning menginstruksikan sistem mengabaikan folder wilayah lainnya secara instan.

- 13) hitung arus antar zona (0.4693 s): DuckDB mengompilasi jutaan rekaman menjadi tabel arus ringkas hanya dalam kurun setengah detik.
- 14) 2 langkah graph (NetworkX) (0.0067 s): Penelusuran jaringan (graph) untuk dua langkah sangat kilat karena murni menggunakan referensi relasi antar-node di RAM.
- 15) 2 langkah SQL self-join (DuckDB) (0.0181 s): Penggabungan tabel terhadap dirinya sendiri (self-join) berlangsung efisien walau sedikit tertinggal dibanding penelusuran graph murni.
- 16) cara 1 loop Python, 5 kotak (12.9721 s): Pengecekan titik spasial secara manual (loop) sangat tidak direkomendasikan karena baru 5 kotak saja sudah menguras nyaris 13 detik.
- 17) cara 2 NumPy, 1000 kotak (0.5644 s): Eksekusi 1000 area menjadi jauh lebih ringan berkat kemampuan kalkulasi komputasi vektor (vectorized) dari NumPy secara paralel.
- 18) bangun indeks STRtree (0.0787 s): Pembuatan struktur hierarki pohon spasial (R-tree) di awal hanya mengorbankan waktu sepersekian detik.
- 19) cara 3 STRtree, 1000 kotak (0.0081 s): Indeks spasial secara brutal menyingkirkan metode lain berkat kemampuannya membuang miliaran kemungkinan area irisan yang tak bersinggungan (pruning) secara instan.

Table k-9

| Kebutuhan                                    | Cara yang unggul                | Cara yang lemah              | Angka Anda                                                                                                                       |
|----------------------------------------------|---------------------------------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Ambil satu data lewat id                     | Key-value; tabel dengan indeks  | Tabel tanpa indeks           | Key-Value (0.0015 s) & Indeks (0.0044 s) vs Tanpa Indeks (7.1963 s)                                                              |
| Menghitung atau merangkum banyak baris       | Per kolom (DuckDB pada Parquet) | Key-value (buka semua nilai) | DuckDB (0.3343 s) vs Key-Value (1.3240 s) / SQLite (1.9939 s)                                                                    |
| Data dengan field opsional atau berubah-ubah | Document                        | Tabel (kolom NULL, migrasi)  | (kualitatif) Dokumen baru menerima sisipan field "cuaca" dengan mulus tanpa mengubah struktur dokumen lama yang tak memilikinya. |
| Keterhubungan dengan jumlah langkah bebas    | Graph                           | SQL (satu JOIN per langkah)  | Graph (0.0067 s) vs SQL Self-join (0.0181 s)                                                                                     |

|                                               |                       |                       |                                                                                                                                         |
|-----------------------------------------------|-----------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Mencari lokasi dalam wilayah                  | Spasial dengan indeks | Memeriksa semua titik | STRtree (0.0081 s untuk 1000 kotak) vs Loop (12.9721 s untuk 5 kotak saja)                                                              |
| Perubahan yang harus aman dan bisa dibatalkan | Tabel Delta           | Folder Parquet biasa  | (kualitatif) Sistem kebal error, menolak paksa modifikasi "kolom asing" (SchemaMismatchError) dan mendukung Time Travel ke masa lampau. |

2. Data lain, cara lain. Pilih satu data lain (mis. “zona”, “hari”, atau “metode bayar”) dan rancang tiga cara menyimpannya (tabel, document, dan graph atau key-value). Untuk masing-masing, tulis satu pertanyaan yang mudah dan satu yang canggung dijawab.

```

data_tugas = sample[["id_perjalanan", "total_bayar", "metode_bayar", "tip"]].head(1000).copy()

# 1. MODEL TABEL (SQLite)
con_tugas = sqlite3.connect(":memory:") # Menggunakan memori sementara
data_tugas.to_sql("pembayaran", con_tugas, index=False)

mudah_sql = con_tugas.execute("SELECT metode_bayar, AVG(tip) FROM pembayaran GROUP BY metode_bayar").fetchall()
print("SQL Mudah (Agregasi):", mudah_sql)

# 2. MODEL DOCUMENT (TinyDB)
from tinydb import TinyDB
db_tugas = TinyDB(storage=MemoryStorage)
tabel_doc = db_tugas.table("pembayaran")
dokumen_bayar = data_tugas.to_dict(orient="records")
tabel_doc.insert_multiple(dokumen_bayar)

mudah_doc = len(tabel_doc.search(Query().metode_bayar == "Kartu kredit"))
print("Document Mudah (Pencarian fleksibel):", mudah_doc)

# 3. MODEL KEY-VALUE (Kamus Python)
kv_tugas = {f"bayar:{r['id_perjalanan']}": r for r in data_tugas.to_dict(orient="records")}

mudah_kv = kv_tugas["bayar:10"]
print("Key-Value Mudah (Ambil via ID):", mudah_kv)
canggung_kv = [v for k, v in kv_tugas.items() if v["metode_bayar"] == "Tunai"]

...
SQL Mudah (Agregasi): [(None, 3.632068965517241), ('Gratis', 0.0), ('Kartu kredit', 4.261797175866495), ('Sengketa', 0.0)]
Document Mudah (Pencarian fleksibel): 779
Key-Value Mudah (Ambil via ID): {'id_perjalanan': 10, 'total_bayar': 15.0, 'metode_bayar': 'Kartu kredit', 'tip': 1.0}

```

3. Delta Lake dua hari. Simulasikan dua hari kerja: hari pertama menulis sebagian data, hari kedua menambah data dan mengoreksi beberapa baris. Tunjukkan bahwa Anda bisa membaca tabel seperti keadaannya di akhir hari pertama, dan lampirkan isi `_delta_log/`.



layanan dan kecepatan akses dengan membiarkan kondisi data terdistribusi pada akhirnya seragam, sehingga pembaca berpeluang mendapatkan sisa rekaman memori data versi sebelumnya dalam waktu sesaat. Teorema CAP merupakan teori dasar terdistribusi bahwa apabila koneksi jaringan antara beberapa mesin peladen terputus secara fisik, sistem dipaksa mengalah dan hanya boleh memilih salah satu antara mempertahankan keseragaman atau ketersediaan. Sebagai contoh untuk aplikasi taksi, data saldo pembayaran penumpang menuntut keseragaman agar tidak ada saldo pelanggan yang terpotong secara ganda akibat aplikasi membaca nilai saldo usang. Sebaliknya, fitur daftar perjalanan terbaru pada layar dasbor sopir dapat dikorbankan agar sedikit tertinggal pembaruannya demi memilih ketersediaan supaya layar aplikasi tidak membeku jika sinyal sedang melemah.

3) Kecepatan dan pengorbanan pada sistem key-value.

Pencarian satu data melalui identitas tunggal sangat gesit lantaran struktur sistem penyimpanannya pada hakikatnya bertindak sebagai indeks absolut itu sendiri. Sistem akan mengumpulkan data-data terkait memutar kata kunci yang ada, lalu kunci ini akan membimbing komputer merujuk langsung ke letak absolut data dan melampaui keharusan menelusuri baris per baris. Sebaliknya, fungsi perhitungan numerik dan merangkum sangat melambat pada sistem ini karena nilai di dalam penyimpanannya tidak dipilah ke format kolom melainkan hanya diamankan sebagai seonggok dokumen teks. Saat menghitung, logika komputer akan terhambat oleh rutinitas memecahkan teks dan menggali nilai di dalamnya berulang-ulang dari awal. Pengorbanan utama dari wujud kesederhanaan sistem ini adalah hilangnya kemampuan meninjau atau mencari data dengan berpatokan dari isi atau nilai datanya.

4) Penyatuan dan rujukan pada model dokumen.

Menyatukan data merupakan pendekatan yang menggabungkan semua ragam informasi relevan agar bersarang di dalam satu struktur induk dokumen secara bersama-sama. Pendekatan ini mempercepat laju pembacaan dan cocok bagi karakter kumpulan data yang harus disajikan secara lengkap sekaligus namun jarang sekali melalui pembaharuan nilai. Di sisi lain, metode merujuk bekerja dengan cara dokumen induk hanya menyimpan dan menanam nilai identitas dari entitas, sementara rincian profil identitas tersebut diamankan di arsip entitas terpisah. Pendekatan ini lebih menghemat volume simpan memori disk namun harus menarik identitas tambahan setiap kali dipanggil. Model rujukan ini sangat tepat dipergunakan saat menangani entitas profil yang sifat datanya sangat dinamis atau atribut datanya sering direplikasi ke sistem dokumen eksternal lainnya.

5) Keunggulan jaringan untuk pertanyaan keterhubungan di luar metrik kecepatan.

Sistem relasi jaringan sangat melampaui efektivitas eksekusi standar SQL pada tipe pertanyaan tentang ekskalasi keterhubungan, eksplorasi rekomendasi

berantai, pelacakan rute lintasan, maupun jangkauan koneksi simpul bebas. Alasan utama di luar metrik kecepatan ada pada kemudahan rancangan penulisan kuerinya. Untuk menaikkan rasio lompatan dari satu langkah ke beberapa langkah berikutnya, logika SQL menuntut operator penulisan sintaks penggabungan diri yang berulang-ulang dan sangat rumit. Pada pengoperasian instruksi basis data relasi jaringan, perluasan kedalaman jangkauan dari awal rute dapat diakomodasi secara murni hanya dengan mengubah batas angka pada parameter instruksi, yang pada akhirnya sangat mempermudah perawatan skrip seiring berjalannya proyek.

- 6) Cara kerja indeks spasial dan alasan klaim kecepatannya bisa menyesatkan.  
Indeks spasial meringankan beban pelacakan radius murni dengan mengkategorikan gugusan titik di permukaan yang memiliki kedekatan geografis ke dalam pola selubung kotak berjenjang. Kotak-kotak induk di luar wilayah pencarian ini akan dikesampingkan seketika sehingga menyerupai logika pemangkas partisi. Sementara itu, narasi pemasaran atau klaim bahwa suatu indeks seribu kali lebih cepat merupakan pernyataan yang cacat makna tanpa menyebut standar acuan pembandingan mula-mulanya. Percepatan algoritma hingga ribuan kali lipat hanya bisa dipenuhi asalkan sistem disandingkan bersebelahan dengan eksekusi sekuensial atau perulangan manual yang primitif. Perbandingan waktu yang sama jika disandingkan dengan barisan komputasi matriks yang canggih umumnya hanya akan menghasilkan selisih hitungan satuan hingga puluhan kali lebih laju.
- 7) Log transaksi Delta Lake dan bahaya pembersihan retensi nol jam.  
Direktori log transaksi pada arsitektur repositori Delta dipenuhi dokumen pelacak format JSON yang menginventarisasi wujud baris arsip aktif beserta jejak berkas yang tereliminasi. Penulisan pada sistem ini bersandar pada kelengkapan komit tersebut dan dicatat secara utuh sekaligus eksklusif, yang berarti pencatatan harus berhasil penuh atau ditolak penuh tanpa meninggalkan versi operasi setengah jalan. Fitur pemulihan berkala ke masa lalu bisa terjadi murni karena prosedur perubahan pemadatan tak akan menyalakan berkas lawas melainkan hanya mendata arsip segar dan melepaskan wewenang arsip kelam dari barisan versi aktif yang baru. Perintah pembersihan berkas secara utuh dengan retensi murni bernilai nol jam sangat berisiko menghapus fisik arsip usang di memori mesin yang berpotensi sedang dibaca oleh siklus program pelacak yang tertinggal, serta menghentikan mutlak kemampuan tabel tersebut untuk memundurkan kembali kondisinya ke riwayat masa lalunya.